Prototype

Matthew Goldenberg, Maria Woods

2026-10-30

Initial Generation of the Virginia Dataset.

Install Dependencies

This code requires the following packages to run successfully: tidyverse, data.table, sf, tigris. If this is your first time using R and this code, switch eval: false to be eval: true and run the code to install these packages. Then switch this back to eval: false.

```
# Install any missing packages once if needed:
install.packages(c("tidyverse", "data.table", "sf", "tigris"))
```

Loads the needed R Libraries

```
library(tidyverse)
library(data.table)
library(sf)
library(tigris)
options(tigris_use_cache = TRUE)
```

How to correctly download the initial data.

To correctly generate the initial data set to use in the follow up charts and graphs follow these directions.

1. Create a folder called Data and a Folder Called Scripts.
2. Capitalization Matters, both folders should start with a capital letter.

3. These two folders should be subfolders in the same parent folder and the parent folder can be called whatever you want the parent folder to be called.
4. Save this QMD file in the folder called Scripts.
5. Navigate to this website: https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents?resource=download
6. Click download in the upper right hand corner of the website.
7. Click download dataset as zip.
8. Click the file that opens.
9. Click extract all.
10. Extract it to the folder you created labeled Data so that the code can work.

Extraction of Virginia from the full dataset.

The below code extracts from the full data set file those rows that have the value under "State" = VA. This is in an effort to try to get a Virginia data set only to draw graphs and run code on.

```
# Bring Full table in
dffull <- fread("../Data/US_Accidents_March23.csv") %>%
  as_tibble()

# Filter full table as just state
dfva <- dffull %>%
  filter(State == "VA")
# Write VA Only Dataset------------------------------
write_csv(dfva, "../Data/Virginia_Accidents.csv")
```

Validation Code of Virginia Data Set Against Full Data Set

The two errors to guard against here are:

1. In the underlying data set, the authors can always choose to add additional columns to the data set to explore different effects on traffic. If you do not run the set up code and just use the Virginia data set only without checks, you might miss the inclusion of these additional columns when the code is run. For example, if you use a previous saved down version and don't check that the columns of the full data set = the columns of the reduced VA, the author's might change something by adding a column to their data set and you wouldn't know.
2. The authors of the data set can always choose to be inconsistent and abbreviate Virginia as VA, va, or type the full name of Virginia out. Since our analysis is based on Virginia data only, we want to make sure when we pull Virginia out of the full data set that we

are getting all Virginia records, and not just the part with only the abbreviation VA. We also want to make sure we are getting all columns to start building the data set. This validation code checks for both parts by setting up the test and then setting up a falsification procedure as well (i.e. if the test passes everything, how do we know our test even does the correct job). Then to perform effective memory handling, at the end it kills the corrupt data frames in the R image that were used in the falsification procedure, and the full data set once the Virginia data set has been written to /Data.This leaves only dfva in the image space as a Virginia only data set.

```
# Verify that the data brought in has the same dimensions as the full dataset
check_col <- function(x, y){
  stopifnot(ncol(x) == ncol(y))
  invisible(TRUE)
}

# create corruption
set.seed(123)
dfvacor <- dfva %>%
  mutate(corcol = rnorm(nrow(dfva), mean = 50, sd = 10))

# Verify works on original and fails on corrupted.
c(passes_originalnew = !inherits(try(check_col(dffull, dfva),
                                     silent = TRUE), "try-error"),
  rejects_corruptnew =  inherits(try(check_col(dffull, dfvacor),
                                     silent = TRUE), "try-error"))
```

```
passes_originalnew rejects_corruptnew
              TRUE                TRUE
```

```
#---------------------
### --- Verify ALL Virginia observations were pulled in
#(Completeness & Alias Check)
check_va_complete <- function(full_states, y) {
  # 1. Count exact "VA" matches
  expected_n <- sum(full_states == "VA", na.rm = TRUE)

  # 2. Extract unique states first (massive speed improvement -
  # only ~50 strings to clean instead of millions)
  unique_states <- unique(full_states)
  clean_unique <- gsub("\\.", "", trimws(toupper(unique_states)))

  # 3. Find which unique state values match "VA" or "VIRGINIA"
```

```
  # and count their total occurrences
  va_variants <- unique_states[clean_unique %in% c("VA", "VIRGINIA")]
  clean_n <- sum(full_states %in% va_variants, na.rm = TRUE)

  stopifnot(
    !anyNA(y$State),
    all(y$State == "VA"),
    nrow(y) == expected_n,
    expected_n == clean_n
  )
  invisible(TRUE)
}

# Create corruption 1: Simulate dropping 10 rows from dfva
dfva_incomplete <- dfva %>%
  slice(-c(1:10))

# Create corruption 2: Use a TINY mock dataset instead of
#duplicating dffull to save RAM
mock_full_states <- c("MD", "NY", "VA", "VA", "Virginia", "va")
mock_dfva_bad <- data.frame(State = c("VA", "VA"))
# Misses the "Virginia" and "va" variants

# Verify works on original and fails on both corruptions
c(passes_original        = !inherits(try(check_va_complete(dffull$State, dfva),
                                         silent = TRUE), "try-error"),
  rejects_dropped_rows   =  inherits(try(check_va_complete(dffull$State, dfva_incomplete),
                                         silent = TRUE), "try-error"),
  rejects_virginia_spell =  inherits(try(check_va_complete(mock_full_states, mock_dfva_bad),
                                         silent = TRUE), "try-error"))
```

```
      passes_original   rejects_dropped_rows rejects_virginia_spell
                 TRUE                   TRUE                   TRUE
```

```
# Remove Unneeded memory from R Global Environment.----------------------
rm(dfvacor, dffull, dfva_incomplete, mock_full_states, mock_dfva_bad)
```

Attempting Join of the Virginia Dataset with the US Census Bureau data.

This code realizes that the Virginia Dataset by itself will unfairly bias results to the more populous counties as the "most accidents" are likely to occur in the biggest counties by virtue

of the fact that there are just simply more people on the road. So we thought it might be helpful to try to link the current dataset using a join to a Census Bureau population dataset for Virginia counties.We can then weight our count of accidents by the population in each county to show a rate instead of a raw count of accidents per county.

Our dataset from kaggle offers two possibilities to join to a population data set. First, our dataset has "Counties" as a column variable. It should be possible, but complicated, to do text editing and link this column to the Census data. Second, our dataset has geolocation data such as longitude and latitude of the accident. Because analyzing counties was important to us with our question, we choose the first method as it seemed to have a stronger tie to the original county column in the dataset and the exact geo-location of the accident may not be a 1-1 tie to the official US Census Bureau county name given in our Kaggle dataset.Then we used the geolocation dataset as a verification check with the idea that if there is a low disagreement between what the geolocated data of the accident would say regarding the location of the accident, and the county name, then we can trust the population values we are drawing from the Census Bureau. The verification check also made sure using anti_join, that we were not missing any Virginia populations.

Finally, at the end, un-needed dataframes were removed from the R image for efficient memory storage.

```
# ============================================================================
# FETCH & CLEAN OFFICIAL US CENSUS POPULATION DATA FOR VIRGINIA
# ============================================================================

dfva<-read.csv("../Data/Virginia_Accidents.csv")

census_url <- "https://www2.census.gov/programs-surveys/popest/datasets/2020-2023/counties/t

va_pop <- read_csv(
  census_url,
  locale = locale(encoding = "latin1"),
  show_col_types = FALSE
) %>%
  filter(STNAME == "Virginia", SUMLEV == "050") %>%
  # 050 = All 133 VA Counties & Independent Cities
  transmute(
    GEOID       = paste0(STATE, COUNTY),
    census_name = str_squish(CTYNAME),
    census_key  = str_to_lower(census_name),
    population  = POPESTIMATE2023
  )

# ============================================================================
```

```r
#  HARMONIZE VIRGINIA COUNTY & INDEPENDENT CITY NAMES
# ============================================================================
# The 34 VA independent cities that do NOT share a name with a surrounding county
unambiguous_va_cities <- c(
  "alexandria", "bristol", "buena vista", "charlottesville", "chesapeake",
  "colonial heights", "covington", "danville", "emporia", "falls church",
  "fredericksburg", "galax", "hampton", "harrisonburg", "hopewell",
  "lexington", "lynchburg", "manassas", "manassas park", "martinsville",
  "newport news", "norfolk", "norton", "petersburg", "poquoson",
  "portsmouth", "radford", "salem", "staunton", "suffolk",
  "virginia beach", "waynesboro", "williamsburg", "winchester"
)

dfva_clean <- dfva %>%
  mutate(
    # 1. Standardize encoding,
    #remove parentheses around "(city)" or "(county)", and lowercase
    cnty_raw   = str_to_lower(str_squish(iconv(County, to = "UTF-8", sub = ""))),
    cnty_clean = str_squish(str_replace_all(cnty_raw, "[()]", "")),

    # 2. Extract base name by stripping trailing " county" or " city"
    cnty_base  = str_remove(cnty_clean, "\\s+(county|city)$"),

    # 3. Map every jurisdiction 1-to-1 from the County column to the Census CTYNAME format
    census_key = case_when(
      # Fix the 2 Virginia counties that literally have "City" in their county name
      cnty_base %in% c("james", "james city") ~ "james city county",
      cnty_base %in% c("charles", "charles city") ~ "charles city county",

      # Bedford city merged into Bedford County in 2013
      cnty_base == "bedford" ~ "bedford county",

      # Resolve the 4 VA County vs. Independent City collisions
        #(Fairfax, Roanoke, Richmond, Franklin)
      # Because raw independent cities are labeled
        #"(city)" or "city", keep them as "city",
        # and map plain "Fairfax" / "Roanoke" / "Richmond" /
        #"Franklin" directly to "county"
      cnty_clean %in% c("fairfax city", "roanoke city",
                        "richmond city", "franklin city") ~ cnty_clean,
      str_detect(cnty_clean, "^city of (fairfax|roanoke|richmond|franklin)$") ~ paste0(str_r
      cnty_base %in% c("fairfax", "roanoke", "richmond", "franklin")
```

```
      ~ paste0(cnty_base, " county"),

      # Append " city" to the 34 unambiguous independent cities
      cnty_base %in% unambiguous_va_cities
      ~ paste0(cnty_base, " city"),

      # Append " county" to all remaining Virginia counties
      !is.na(cnty_base) & cnty_base != "" ~ paste0(cnty_base, " county"),
      TRUE ~ NA_character_
    )
  )



#---------------------------------

# Verify 100% match rate against Census
unmatched <- dfva_clean %>%
  anti_join(va_pop, by = "census_key") %>%
  count(County, City, census_key, sort = TRUE)

print(unmatched)
```

```
[1] County     City       census_key n
<0 rows> (or 0-length row.names)
```

```
# 2. Left join population data onto the accident records
  # and remove helper columns
dfva_joined <- dfva_clean %>%
  left_join(va_pop, by = "census_key") %>%
  select(-cnty_raw, -cnty_clean, -cnty_base)

# ============================================================================
# STEP 5 SPATIAL AUDIT COMPARING COUNTY TEXT VS. GPS COORDINATES
# ============================================================================


# cb = FALSE uses full legal boundaries
#(including bridges over water in Hampton Roads/Arlington)
va_polygons <- counties(state = "VA", cb = FALSE, year = 2022) %>%
  transmute(
```

```
    spatial_GEOID = GEOID,
    spatial_name  = NAMELSAD,
    spatial_key   = str_to_lower(str_squish(NAMELSAD))
  )

dfva_audit <- dfva_clean %>%
  filter(!is.na(Start_Lng), !is.na(Start_Lat)) %>%
  st_as_sf(coords = c("Start_Lng", "Start_Lat"),
           crs = 4326, remove = FALSE) %>%
  st_transform(st_crs(va_polygons)) %>%
  st_join(va_polygons, join = st_within) %>%
  st_drop_geometry()

audit_summary <- dfva_audit %>%
  summarise(
    total_rows       = n(),
    exact_matches    = sum(census_key == spatial_key, na.rm = TRUE),
    mismatches       = sum(census_key != spatial_key, na.rm = TRUE),
    outside_va_poly  = sum(is.na(spatial_key)),
    match_percentage = round((exact_matches / total_rows) * 100, 2)
  )

print(audit_summary)
```

```
  total_rows exact_matches mismatches outside_va_poly match_percentage
1     303307        300277       2985              45               99
```

```
mismatch_breakdown <- dfva_audit %>%
  filter(census_key != spatial_key | is.na(spatial_key)) %>%
  count(County, City, text_assigned = census_key,
        gps_assigned = spatial_key, sort = TRUE)

# Summary Table Formulations -----
dfva_county_summary <- dfva_joined %>%
  group_by(GEOID, census_name, population) %>%
  summarise(
    total_accidents  = n(),
    severe_accidents = sum(Severity >= 3, na.rm = TRUE),
    avg_severity     = mean(Severity, na.rm = TRUE),
    .groups = "drop"
  ) %>%
```

```
  mutate(
    accidents_per_person = total_accidents / population,
    accidents_per_100k   = (total_accidents / population) * 100000,
    severe_per_100k      = (severe_accidents / population) * 100000
  ) %>%
  arrange(desc(accidents_per_100k))

## Write the joined dataset
# Save full row-level Virginia dataset with Census population attached
write_csv(dfva_joined, "../Data/Virginia_Accidents_With_Pop.csv")
##Remove Unneeded Memory from R image-------------------------------------
rm(dfva, dfva_clean, dfva_audit, dfva_joined)
```

Validation of Analysis Data Set only

Since accident Severity is a main portion of our analysis, this validation check looks at accident Severity and makes sure that there are no NA's or values off the expected accident Severity scale of 1-4. This tests the data dictionary on Kaggle for this column where it describes Severity as:

Shows the severity of the accident, a number between 1 and 4, where 1 indicates the least impact on traffic (i.e., short delay as a result of the accident) and 4 indicates a significant impact on traffic (i.e., long delay).

```
dfva_joined <- read.csv("../Data/Virginia_Accidents_With_Pop.csv")

#Check Accident Severity
check_severity <- function(x){
  stopifnot(sum(unique(x$Severity))<=10,
            !any(is.na(x$Severity)))
  invisible(TRUE)
}

#data corruption insertion of severity 5
dfvacorrupt<-data.frame(Severity = 1:5, Random = 1:5)
#data corruption, insertion of NA values.
dfvacorrupt1<-data.frame(Severity = as.numeric(c(1,2,"",4, 4)), Random = 1:5)

#Checks Severity
c(passes_original        = !inherits(try(check_severity(dfva_joined),
                                         silent = TRUE), "try-error"),
```

```
  rejects_corrupt   =  inherits(try(check_severity(dfvacorrupt),
                                    silent = TRUE), "try-error"))
```

```
passes_original rejects_corrupt
           TRUE            TRUE
```

```
#Checks NA
c(passes_original        = !inherits(try(check_severity(dfva_joined),
                                    silent = TRUE), "try-error"),
  rejects_corrupt   =  inherits(try(check_severity(dfvacorrupt1),
                                    silent = TRUE), "try-error"))
```

```
passes_original rejects_corrupt
           TRUE            TRUE
```

Graphs and Summary Tables

```
###---------------------- Graphing
dfva_joined <- read.csv("../Data/Virginia_Accidents_With_Pop.csv")

acc<-dfva_joined

summary(acc)
```

```
      ID              Source             Severity      Start_Time
 Length:303301      Length:303301      Min.   :1.000   Length:303301
 Class :character   Class :character   1st Qu.:2.000   Class :character
 Mode  :character   Mode  :character   Median :2.000   Mode  :character
                                       Mean   :2.279
                                       3rd Qu.:2.000
                                       Max.   :4.000

   End_Time           Start_Lat       Start_Lng         End_Lat
 Length:303301      Min.   :36.54   Min.   :-83.58   Min.   :36.53
 Class :character   1st Qu.:37.31   1st Qu.:-77.61   1st Qu.:37.30
 Mode  :character   Median :37.63   Median :-77.43   Median :37.69
                    Mean   :37.90   Mean   :-77.61   Mean   :37.91
                    3rd Qu.:38.77   3rd Qu.:-77.19   3rd Qu.:38.76
```

```
                  Max.   :39.43   Max.   :-75.38   Max.   :39.43
                                                  NA's   :98128
   End_Lng        Distance.mi.     Description          Street
Min.   :-83.51   Min.   : 0.0000   Length:303301      Length:303301
1st Qu.:-77.68   1st Qu.: 0.0000   Class :character   Class :character
Median :-77.44   Median : 0.1350   Mode  :character   Mode  :character
Mean   :-77.64   Mean   : 0.7621
3rd Qu.:-77.19   3rd Qu.: 0.8500
Max.   :-75.37   Max.   :71.6550
NA's   :98128
    City              County             State              Zipcode
Length:303301      Length:303301      Length:303301      Length:303301
Class :character   Class :character   Class :character   Class :character
Mode  :character   Mode  :character   Mode  :character   Mode  :character


  Country            Timezone          Airport_Code       Weather_Timestamp
Length:303301      Length:303301      Length:303301      Length:303301
Class :character   Class :character   Class :character   Class :character
Mode  :character   Mode  :character   Mode  :character   Mode  :character


Temperature.F.   Wind_Chill.F.     Humidity...      Pressure.in.
Min.   :-11.00   Min.   :-32.00   Min.   :  1.00   Min.   : 0.30
1st Qu.: 45.00   1st Qu.: 40.70   1st Qu.: 49.00   1st Qu.:29.64
Median : 60.10   Median : 59.00   Median : 68.00   Median :29.89
Mean   : 59.24   Mean   : 56.97   Mean   : 67.26   Mean   :29.74
3rd Qu.: 73.00   3rd Qu.: 73.00   3rd Qu.: 88.00   3rd Qu.:30.07
Max.   :162.00   Max.   :162.00   Max.   :100.00   Max.   :30.85
NA's   :17649    NA's   :62844    NA's   :18542    NA's   :17303
Visibility.mi.   Wind_Direction     Wind_Speed.mph.  Precipitation.in.
Min.   : 0.000   Length:303301      Min.   : 0.000   Min.   : 0.000
1st Qu.:10.000   Class :character   1st Qu.: 3.000   1st Qu.: 0.000
Median :10.000   Mode  :character   Median : 6.900   Median : 0.000
Mean   : 9.134                      Mean   : 6.922   Mean   : 0.008
3rd Qu.:10.000                      3rd Qu.: 9.200   3rd Qu.: 0.000
Max.   :10.000                      Max.   :66.700   Max.   :23.970
NA's   :17431                       NA's   :29810    NA's   :66244
Weather_Condition   Amenity           Bump          Crossing
```

```
Length:303301      Mode :logical   Mode :logical   Mode :logical
Class :character   FALSE:299927    FALSE:303265    FALSE:280833
Mode  :character   TRUE :3374      TRUE :36        TRUE :22468
```

```
 Give_Way        Junction        No_Exit         Railway
Mode :logical   Mode :logical   Mode :logical   Mode :logical
FALSE:300988    FALSE:282409    FALSE:302360    FALSE:302271
TRUE :2313      TRUE :20892     TRUE :941       TRUE :1030
```

```
Roundabout       Station           Stop         Traffic_Calming
Mode :logical   Mode :logical   Mode :logical   Mode :logical
FALSE:303291    FALSE:300278    FALSE:298347    FALSE:303143
TRUE :10        TRUE :3023      TRUE :4954      TRUE :158
```

```
Traffic_Signal  Turning_Loop    Sunrise_Sunset     Civil_Twilight
Mode :logical   Mode :logical   Length:303301      Length:303301
FALSE:266899    FALSE:303301    Class :character   Class :character
TRUE :36402                     Mode  :character   Mode  :character
```

```
Nautical_Twilight  Astronomical_Twilight  census_key            GEOID
Length:303301      Length:303301         Length:303301      Min.   :51001
Class :character   Class :character      Class :character   1st Qu.:51059
Mode  :character   Mode  :character      Mode  :character   Median :51107
                                                            Mean   :51260
                                                            3rd Qu.:51550
                                                            Max.   :51840
```

```
census_name          population
Length:303301      Min.   :   2339
Class :character   1st Qu.:  82654
Mode  :character   Median : 229247
```

```
                  Mean   : 345572
                  3rd Qu.: 436347
                  Max.   :1141878
```

```r
table(acc$Severity)
```

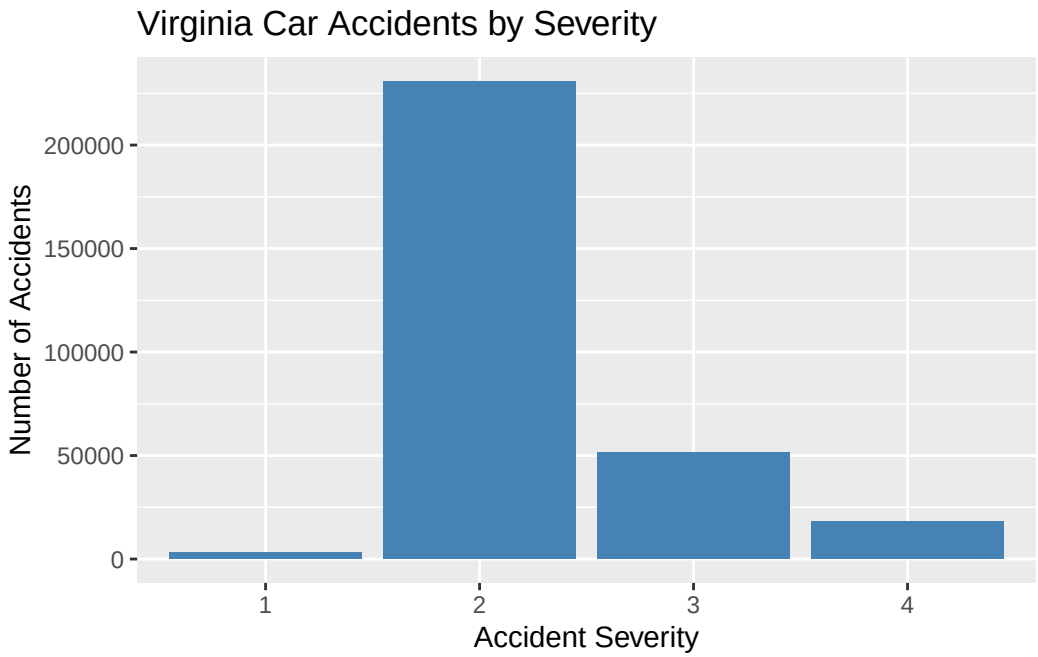
```
     1      2      3      4
  3115 230660  51324  18202
```

```r
summary(acc$`Precipitation(in)`)
```

```
Length  Class   Mode
     0   NULL   NULL
```

```r
acc <- acc %>% mutate(severe = Severity >= 3)

acc %>%
  filter(!is.na(Severity)) %>%
  count(Severity) %>%
  ggplot(aes(x = factor(Severity), y = n)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Virginia Car Accidents by Severity",
    x = "Accident Severity",
    y = "Number of Accidents")
```

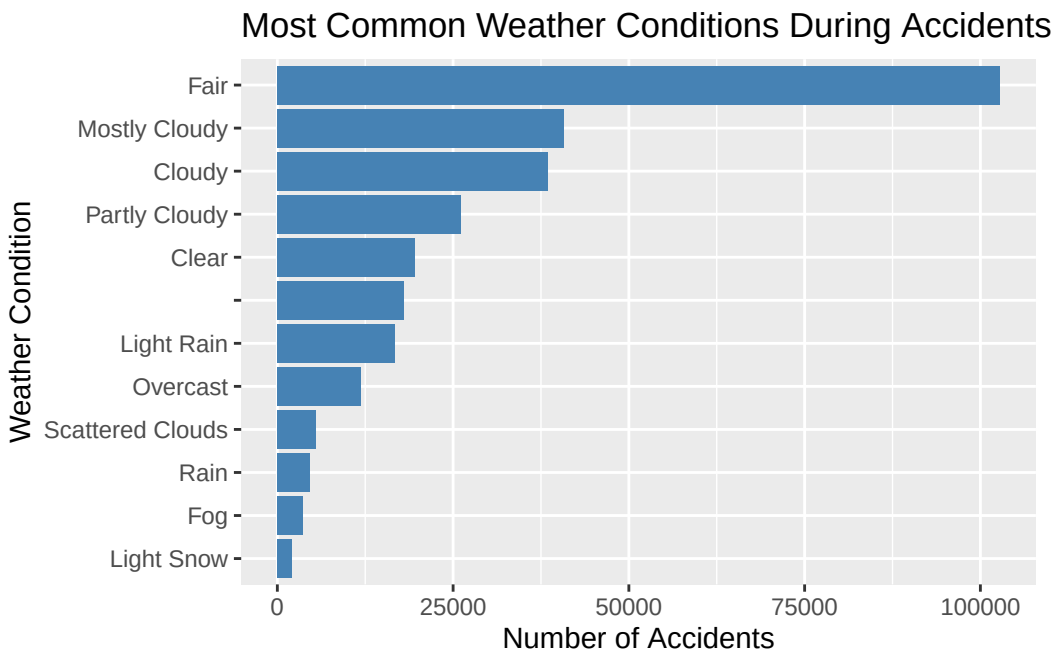



```
acc %>%
  summarise(
    total_accidents = n(),
    severe_accidents = sum(severe, na.rm = TRUE),
    severe_percent = mean(severe, na.rm = TRUE) * 100
  )
```

```
  total_accidents severe_accidents severe_percent
1          303301            69526        22.9231
```

```
acc %>%
  filter(!is.na(Weather_Condition)) %>%
  count(Weather_Condition, sort = TRUE) %>%
  slice_head(n = 12) %>%
  ggplot(
    aes(x = reorder(Weather_Condition, n), y = n)
  ) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Most Common Weather Conditions During Accidents",
    x = "Weather Condition",
```

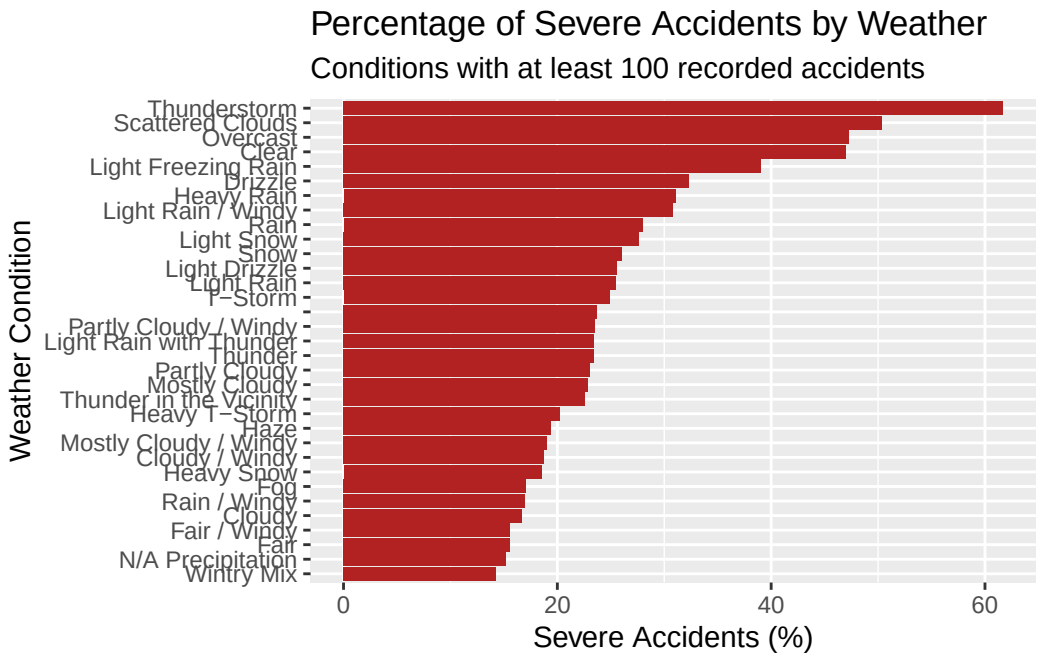
```
    y = "Number of Accidents"
  )
```




```
weather_summary <- acc %>%
  filter(
    !is.na(Weather_Condition),
    !is.na(Severity)
  ) %>%
  group_by(Weather_Condition) %>%
  summarise(
    accidents = n(),
    severe_percent = mean(Severity >= 3) * 100,
    .groups = "drop"
  ) %>%
  filter(accidents >= 100) %>%
  arrange(desc(severe_percent))

weather_summary %>%
  ggplot(
    aes(
      x = reorder(Weather_Condition, severe_percent),
      y = severe_percent
```

```
    )
  ) +
  geom_col(fill = "firebrick") +
  coord_flip() +
  labs(
    title = "Percentage of Severe Accidents by Weather",
    subtitle = "Conditions with at least 100 recorded accidents",
    x = "Weather Condition",
    y = "Severe Accidents (%)"
  )
```

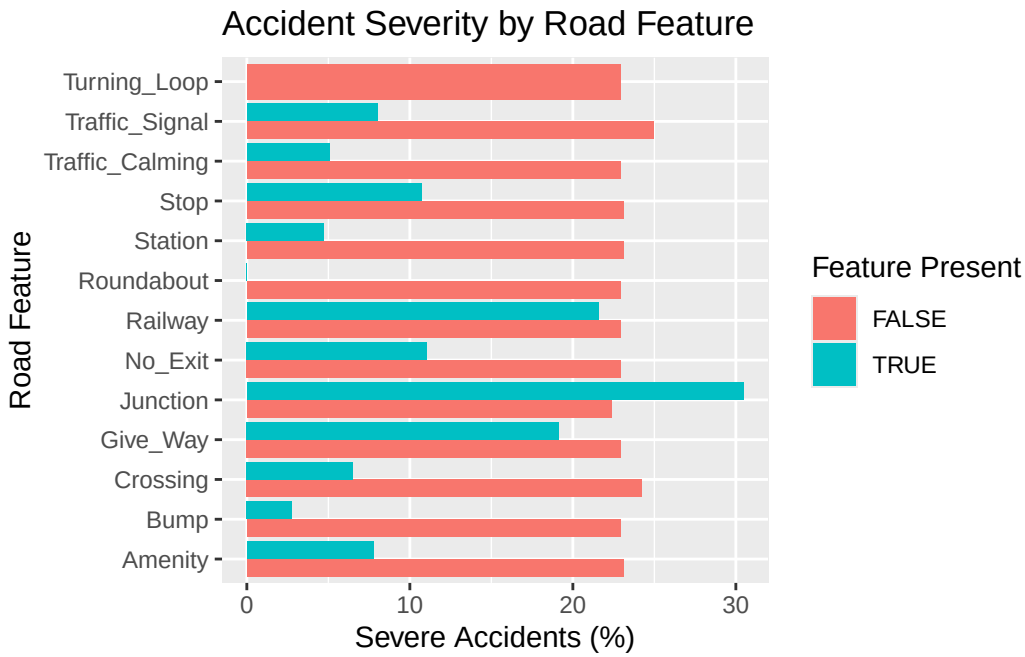



```
road_features <- c(
  "Junction",
  "Crossing",
  "Traffic_Signal",
  "Stop",
  "Railway",
  "Traffic_Calming",
  "Roundabout",
  "Give_Way",
  "Amenity",
  "Bump",
```

```
  "No_Exit",
  "Turning_Loop",
  "Station"
)

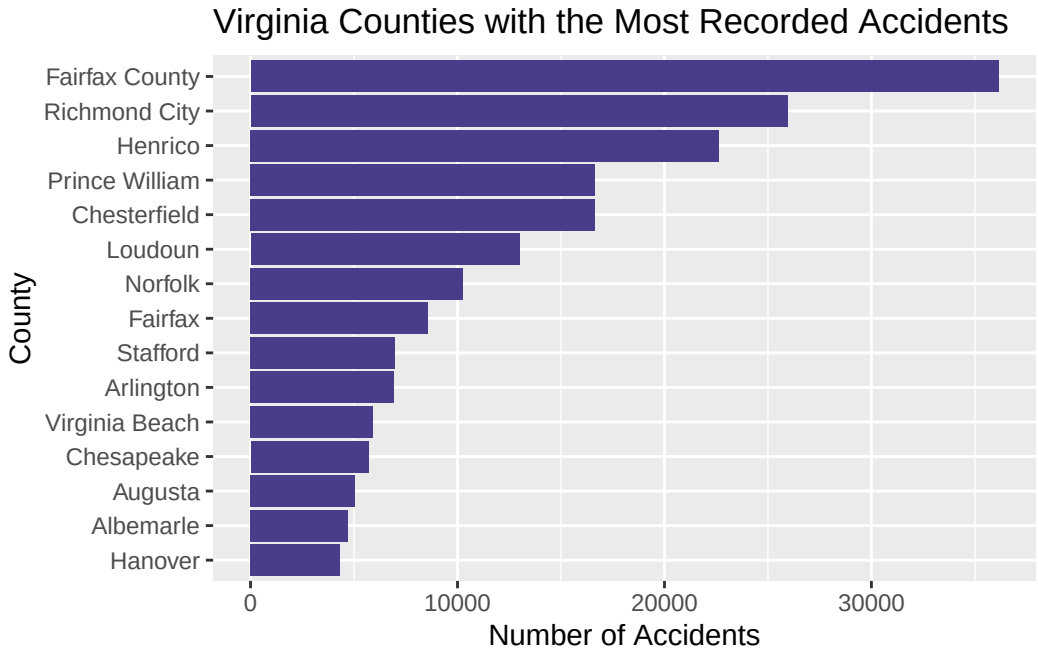
road_summary <- acc %>%
  select(Severity, all_of(road_features)) %>%
  pivot_longer(
    cols = all_of(road_features),
    names_to = "road_feature",
    values_to = "present"
  ) %>%
  filter(!is.na(present), !is.na(Severity)) %>%
  group_by(road_feature, present) %>%
  summarise(
    accidents = n(),
    severe_percent = mean(Severity >= 3) * 100,
    .groups = "drop"
  )

ggplot(
  road_summary,
  aes(
    x = road_feature,
    y = severe_percent,
    fill = factor(present)
  )
) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(
    title = "Accident Severity by Road Feature",
    x = "Road Feature",
    y = "Severe Accidents (%)",
    fill = "Feature Present"
  )
```




```r
county_summary <- acc %>%
  filter(!is.na(County)) %>%
  group_by(County) %>%
  summarise(
    accidents = n(),
    severe_accidents = sum(severe, na.rm = TRUE),
    severe_percent = mean(severe, na.rm = TRUE) * 100,
    .groups = "drop"
  ) %>%
  arrange(desc(accidents))

county_summary %>%
  slice_head(n = 15) %>%
  ggplot(
    aes(x = reorder(County, accidents), y = accidents)
  ) +
  geom_col(fill = "darkslateblue") +
  coord_flip() +
  labs(
    title = "Virginia Counties with the Most Recorded Accidents",
    x = "County",
    y = "Number of Accidents"
  )
```



AI Tracker

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
AI to Help Set Up GitHub Repo	Gemini Pro. 10/19/2026	Step by Step Directions For How to Set Up the Repo Successfully	NA	NA	NA
Forgot how to add a Column to a DataFrame so I could corrupt it for a check	Gemini Pro, 10/25/2026	Suggested to use dplyr mutate function to add the column	Successfully executed dpylr mutate to create corrupted data check.	NA	Only checks if columns are the same count of columns. Doesn't check if any have changed since the last update.

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
Wasn't really sure how to create the check successfully to make sure all VA records from the full dataset were successfully brought over to the VA one. Wanted it to adjust for abbreviations, acronyms etc.So figuring out the string combos, everything to check, etc. got super complicated.	Gemini Pro, 10/25/2026	Created full check.	copied code in	NA	NA
Full check was taking a while to render so wanted to look into whether there was a simpler easier method	Gemini Pro 10/29/26	AI taught me how I could simplify the corrupt data frames and data itself so the validations run on a smaller subset	Used the AI provided Code	Ran Successfully	None

Task	Tool/model & date	AI contribution	Human revision	Verification performed	Residual concern
Join and normalize Virginia traffic accident data with U.S. Census population totals in R	Gemini Pro, October 29, 2026	Developed tidyverse and Base R scripts to download Census data, harmonize Virginia's independent city vs. county naming collisions, and perform a spatial (sf/tigris) vs. text geocoding audit.	User provided dataset structure context, screenshot evidence of (city) formatting and James City/Charles City errors, and a spatial mismatch audit table, driving iterative regex and rule refinement.	AI utilized the ds_python_interpreter to mathematically verify the row sums (1,737 rows) and bidirectional highway border splits from the user's uploaded mismatch screenshot.	Inherent dataset geocoding biases remain: mailing address bleed (e.g., Fairfax accidents labeled Alexandria), enclave city misattribution (Salem vs. Roanoke County), and lack of adjustment for commuter/through-traffic volume.

Individual Tracker

Date	Who Did It	What Was Produced	How Was It Verified
10/29/2026	Matthew	Created QMD	—